

Bilateral Module — Pool Funding Alignment

Frontend Technical Reference (current shipped architecture)

Alliance Research Indicators — STAR client (Angular 19 SPA)

2026-06-17

- 1. Overview & scope
- 2. Where it lives (file map)
- 3. Data model (frontend wire contracts + view-models)
 - 3.1 Catalog read — GET .../hlos-indicators
 - 3.2 Per-result SP picker — GET .../science-programs
 - 3.3 Alignment read-back + write — GET / PATCH .../pool-funding-alignment
 - 3.4 Component-local view-model (not a wire type)
- 4. API contracts consumed
- 5. Services & functions
 - 5.1 BilateralService (root-provided, signal state)
 - 5.2 ApiService (bilateral block)
- 6. Components
 - 6.1 PoolFundingAlignmentComponent (the tab — host/orchestrator)
 - 6.2 SpTocAlignmentBlockComponent (new — pure per-SP cascade)
- 7. Key behaviors & engineering decisions
- 8. Tests
- 9. Removed / retired (for context)
- 10. Open items / follow-ups (not blocking)

1. Overview & scope

This document describes the **Pool Funding Alignment** feature of the bilateral module **as it ships today**, from a frontend technical perspective, for engineers who need to understand each change.

Scope: the Angular 19 SPA only (`research-indicators/`). Data model here means the **TypeScript wire contracts + view-models** the client uses; **database migrations are N/A on the frontend** (no DB — the backend, in a separate repo, owns persistence and migrations).

What the feature does. On an eligible bilateral result, a contributor opens the **Pool Funding Alignment** tab and:

1. answers “Does this result contribute to a Science Program or Accelerator?” (Yes/No);
2. on **Yes**, selects one or more **Science Programs (SPs)** from a per-result picker;
3. for each selected SP, answers “Does this result align with the Program’s ToC indicators?” and, on **Yes**, maps an inline cascade **Level** → **High-Level Output (HLO)** → **Indicator** → **quantitative contribution** sourced from the **lambda-toc** catalog;
4. saves everything in **one request**.

The ToC catalog is a **level-based** model (`OUTPUT` / `OUTCOME` / `EOI`) sourced via the backend from lambda-toc. It replaced an earlier (now retired) **HLO-selection-modal** + (`SP`, `AOW`) -pair flow (see §9).

Constraints honored: Angular 19 + PrimeNG 19, standalone lazy components, signals for state, `MainResponse<T>` envelope through `ApiService`, design tokens only, WCAG 2.1 AA.

2. Where it lives (file map)

All paths under `research-indicators/src/app/`.

File	Lines	Role
<code>shared/interfaces/bilateral/pool-funding-alignment.interface.ts</code>	166	Data model — wire contracts + view-models
<code>shared/services/bilateral.service.ts</code>	~330	Domain service — alignment + catalog state, draft helpers, save
<code>shared/services/api.service.ts</code> (bilateral block)	—	REST methods (<code>bilateralPath</code> + 4 endpoints)
<code>pages/platform/pages/result/pages/pool-funding-alignment/pool-funding-alignment.component.ts</code>	704	The tab page component (host/orchestrator)
<code>.../pool-funding-alignment.component.html</code>	255	Tab template
<code>.../pool-funding-alignment.component.scss</code>	29	Tab styles
<code>.../pool-funding-alignment.component.spec.ts</code>	1403	Tab tests
<code>.../components/sp-toc-alignment-block/sp-toc-alignment-block.component.ts</code>	239	New — pure per-SP cascade block
<code>.../components/sp-toc-alignment-block/sp-toc-alignment-block.component.html</code>	261	Block template

File	Lines	Role
.../components/sp-toc-alignment-block/sp-toc-alignment-block.component.scss	127	Block styles
.../components/sp-toc-alignment-block/sp-toc-alignment-block.component.spec.ts	344	Block tests
testing/toc-catalog.fixture.ts	~980	Canonical Jest fixtures (SP01 live snapshot)
styles/custom-prime-force-styles.scss (bilateral block)	—	Global PrimeNG overlay override for body-appended panels

The tab is a **lazy-loaded standalone component**, conditionally rendered as a result-detail tab (hidden unless `eligible`). No new routes were added (it lives under the existing `/result/:id` tab set).

3. Data model (frontend wire contracts + view-models)

All types live in `pool-funding-alignment.interface.ts`. There are three groups: **read contracts**, **write contracts**, and the **component-local view-model**.

3.1 Catalog read — GET .../hlos-indicators

```
export type TocLevel = 'OUTPUT' | 'OUTCOME' | 'EOI';

export interface TocCatalogIndicator {
  indicator_id: number;
  indicator_description: string;
  unit_of_measurement: string | null; // backend renames upstream `unit_messurament`
  type_value: string | null; // retained for a future indicator-type filter
  target_value: string | null; // backend-resolved for target_year
  target_year: number; // 2026 this cycle
}

export interface TocCatalogResult {
  toc_result_id: number;
  title: string;
  description: string | null;
  aow_code: string | null; // display-only label; null for EOI
  indicators: TocCatalogIndicator[];
}

export interface TocCatalogLevelGroup { level: TocLevel; toc_results: TocCatalogResult[]; }
export interface TocCatalogSp { sp_code: string; levels: TocCatalogLevelGroup[]; }

export interface BilateralTocCatalogResponse {
  result_code: string;
  mapping_status: PoolFundingMappingStatus; // 'mapped' | 'unmapped'
  clarisa_project: PoolFundingClarisaProject | null;
  result_type: string; // backend-owned enum key (e.g. 'capacity_sharing')
  allowed_levels: TocLevel[]; // [] ⇒ hide the ToC cascade
  version_locked: boolean; // true ⇒ read-only (non-2026 live version)
  catalogs: TocCatalogSp[]; // one entry per SP, one per allowed level
}
```

Key semantics: `allowed_levels` is **server-owned** (derived from `result_type`); the client never recomputes it. `allowed_levels: []` hides the ToC block entirely. `version_locked` renders read-only.

3.2 Per-result SP picker — GET .../science-programs

```
export type PoolFundingMappingStatus = 'mapped' | 'unmapped';
export interface PoolFundingClarisaProject { id: number; short_name: string; }

export interface PoolFundingScienceProgram {
  code: string;           // sent back on PATCH sp_codes
  name: string;
  category?: string | null;
  color?: string | null;  // upstream CLARISA fill (not a design token)
  icon_key: string;       // resolves /sps/{icon_key}.png (icon_key === code in current fixtures)
  allocation: number;
}

export interface PoolFundingSciencePrograms {
  result_code: string;
  mapping_status: PoolFundingMappingStatus;
  clarisa_project: PoolFundingClarisaProject | null;
  science_programs: PoolFundingScienceProgram[]; // [] when unmapped
}
```

3.3 Alignment read-back + write — GET / PATCH .../pool-funding-alignment

```
export interface AlignmentResponse {
  result_code: string;
  eligible: boolean;
  has_pool_funding_alignment_eligible: boolean;
  has_contribution: boolean | null;
  selected_science_programs?: AlignmentScienceProgram[];
  selected_levers: AlignmentLever[];
  justification?: string;
  is_synced_to_prms: boolean;
  is_read_only: boolean;
  toc_alignments?: SavedTocAlignment[]; // saved ToC alignments (pre-fill source)
}

export interface UpdatePoolFundingAlignmentDto {
  has_contribution: boolean;
  sp_codes?: string[];
  lever_codes?: string[]; // deprecated, kept for compat
  justification?: string;
  toc_alignments?: TocAlignmentWriteDto[]; // sent together with sp_codes (single save)
}

// FLAT read-back row (decision D-10) – snapshot fields on the row, no wrapper,
// no aow_code, no is_stale. null on "No" rows / when absent.
export interface SavedTocAlignment {
  sp_code: string;
  aligns_with_toc: boolean;
  level: TocLevel | null;
  toc_result_id: number | null;
  indicator_id: number | null;
  quantitative_contribution: number | null;
  toc_result_title: string | null;
  indicator_description: string | null;
  unit_of_measurement: string | null;
  target_value: string | null;
  target_year: number | null;
}

export interface TocAlignmentWriteDto {
  sp_code: string;
  aligns_with_toc: boolean;
  level?: TocLevel;
  toc_result_id?: number;
  indicator_id?: number;
  quantitative_contribution?: number;
}
```

The backend accepts `has_contribution` + `sp_codes` + `toc_alignments[]` in **one PATCH**, validating each alignment's `sp_code` against that same request's SPs — so selection + mapping save in a single pass (no pre-save step).

3.4 Component-local view-model (not a wire type)

```
export interface SpAlignmentDraft {           // one per selected SP, in selection order
  sp_code: string;
  aligns_with_toc: boolean | null;           // null until answered
  level: TocLevel | null;
  toc_result_id: number | null;
  indicator_id: number | null;
  quantitative_contribution: number | null;
}
```

`SpAlignmentDraft` is the editable per-SP working state. `BilateralService.draftsFromSaved()` seeds drafts from `SavedTocAlignment[]`; `writeDtoFromDrafts()` composes the PATCH body from drafts.

Migrations: N/A — frontend has no database. Persistence and migrations are owned by the backend repo.

4. API contracts consumed

All via `ApiService`, returning `MainResponse<T>`; paths built by `bilateralPath(resultCode, suffix)` (`v1/results/{n}/pool-funding-alignment...`, with the `STAR-` prefix stripped from the code).

Method	HTTP / path	Returns
<code>GET_PoolFundingAlignment(resultCode)</code>	GET .../pool-funding-alignment	<code>MainResponse<AlignmentResponse></code> (incl. <code>toc_alignments</code> pre-fill)
<code>GET_PoolFundingSciencePrograms(resultCode)</code>	GET .../science-programs	<code>MainResponse<PoolFundingSciencePrograms></code>
<code>GET_PoolFundingHlosIndicators(resultCode)</code>	GET .../hlos-indicators	<code>MainResponse<BilateralTocCatalogResponse></code> (the level-based catalog)
<code>PATCH_PoolFundingAlignment(resultCode, body)</code>	PATCH .../pool-funding-alignment	<code>MainResponse<AlignmentResponse></code>

A section load issues **3 GETs** (alignment, science-programs, catalog). The catalog is fetched **once per section load**, not per interaction.

Error handling on PATCH: `400` carries `unknown_sp_codes` (rejected SP chips) and `toc_alignments: [{ sp_code, field, message }]` (per-block inline errors); `409` carries synced/PRMS-sourced locks and `toc_mapping_version_locked`; `503` (cold-cache catalog) is surfaced as a catalog error.

5. Services & functions

5.1 `BilateralService` (root-provided, signal state)

Signals (state): `currentAlignment`, `loadingAlignment`, `savingAlignment`, `editable`, `sciencePrograms`, `mappingStatus`, `loadingSciencePrograms`, `tocCatalog`, `loadingTocCatalog`, `tocCatalogError`.

Methods:

Member	Purpose
<code>getAlignment(resultCode)</code>	Loads <code>AlignmentResponse</code> into <code>currentAlignment</code> (keeps prior value on error).
<code>getSciencePrograms(resultCode)</code>	Loads the per-result SP picker list + <code>mappingStatus</code> .
<code>getTocCatalog(resultCode)</code>	Loads the level-based catalog into <code>tocCatalog</code> ; on non-2xx sets <code>tocCatalogError</code> , keeps the prior value (warm-on-error).
<code>catalogForSp(spCode): TocCatalogSp \ null</code>	Pure lookup of an SP's catalog slice from <code>tocCatalog</code> .
<code>draftsFromSaved(saved): SpAlignmentDraft[]</code>	Pre-fill seam — maps saved alignments → per-SP drafts (reads top-level fields only).
<code>writeDtoFromDrafts(drafts): TocAlignmentWriteDto[]</code>	Save seam — emits bare <code>{sp_code, aligns_with_toc:false}</code> for “No”, full DTO for complete “Yes”, omits unanswered/incomplete.
<code>patchAlignment(resultCode, body): PatchAlignmentResult</code>	Save; normalizes the response into a discriminated result.
<code>extractTocAlignmentErrors(errorDetail) (private)</code>	Tolerant parser for the per-alignment 400 payload.

Exported helper types:

```
export interface TocAlignmentError { sp_code: string; field?: string; message: string; }

export type PatchAlignmentResult =
  | { ok: true; data: AlignmentResponse }
  | { ok: false; status: number; description: string;
    fieldErrors?: Record<string, string>;
    unknownSpCodes?: string[];
    tocAlignmentErrors?: TocAlignmentError[]; };
```

5.2 `ApiService` (bilateral block)

Adds `bilateralPath()` + the four methods in §4. `GET_PoolFundingHlosIndicators` is typed to `BilateralTocCatalogResponse` (the reshaped catalog), replacing the retired AOW-pair response type.

6. Components

6.1 `PoolFundingAlignmentComponent` (the tab — host/orchestrator)

Standalone, `OnPush`, `selector: app-pool-funding-alignment`. Local form state:


```
interface AlignmentFormData {
  has_contribution: boolean | null;
  selected_sps: SelectedScienceProgram[];
  toc_drafts: SpAlignmentDraft[]; // keyed by sp_code, selection order
}
```

Local signals: `loadFailed`, `inlineErrors`, `rejectedSpCodes`, `blockErrors` (per-SP 400 map), `versionLockedFrom409`, `formData`.

Computed (derived UI state): `isReadOnly`, `eligible`, `isSyncedToPrms`, `readOnlyCause` (`'synced'` | `'prms-sourced'` | `'permission'` | `null`), `isUnmapped`, `hasNoSciencePrograms`, `showSpPicker`, `showHloSection`, `allowedLevels`, `showTocBlocks` (`allowed_levels.length > 0`), `versionLocked`, `catalogState` (`'loading'` | `'ready'` | `'error'`), `blocksDisabled`, `canSave`, `saveBlockedByIncompleteToc`, `isDirty`, `resultCode`, `staleSnapshots`.

Key methods:

Method	Purpose
<code>seedFromServer</code> / <code>snapshotFromServer</code>	Build <code>formData</code> from <code>AlignmentResponse</code> ; seeds <code>toc_drafts</code> via <code>draftsFromSaved</code> .
<code>onContributionChange</code>	Yes/No on the top-level contribution question.
<code>onSpSelectionChange</code>	On SP multiselect change → clears rejected-code state + <code>queueMicrotask(reconcileDrafts)</code> (deferred to avoid a re-entrant signal write).
<code>reconcileDrafts</code> / <code>syncDraftsToSelection</code>	Add empty drafts for new SPs, drop for removed; destructive removal asks confirm (D-6a).
<code>confirmDestructiveRemoval</code> / <code>applyDestructiveRemoval</code>	<code>ActionsService.showGlobalAlert</code> confirm before discarding a touched/saved SP.
<code>onDraftChange(next)</code>	Upsert a per-SP draft (replace if present, else append) + clear that SP's block error.
<code>draftForSp</code> / <code>catalogForSp</code> / <code>findScienceProgram</code>	Per-SP lookups for block inputs + chip enrichment.
<code>retryCatalog</code>	Re-fetches the catalog (wired to the block's retry).
<code>onSave</code>	Composes the single PATCH (<code>has_contribution</code> + <code>sp_codes</code> + <code>toc_alignments</code> unless version-locked); routes 400 (<code>unknownSpCodes</code> + per-block <code>tocAlignmentErrors</code>) and 409 (<code>toc_mapping_version_locked</code> + <code>synced/PRMS</code>) responses; telemetry <code>bilateral.alignment.saved</code> (+ <code>toc_alignment_count</code>).
<code>isDraftSaveable</code> (<i>private</i>)	A draft is saveable iff answered: <code>unanswered(null)</code> blocks , "No" is complete, "Yes" needs the full cascade (≥ 0 contribution).

The page renders one `app-sp-toc-alignment-block` per selected SP (`@for ... track sp.official_code`) when `showHloSection()` & `showTocBlocks()`; passes `catalog`, `allowedLevels`, `draft`, `disabled`, `inlineErrors`, `catalogState`; handles (`draftChange`) (`upsert`) and (`retryCatalog`).

6.2 `SpTocAlignmentBlockComponent` (new — pure per-SP cascade)

Standalone, `OnPush`, `selector: app-sp-toc-alignment-block`. **Pure/presentational:** it renders from inputs and emits a brand-new draft on every change; it never mutates its inputs (per-SP independence is structural).

Inputs (signal inputs): `sp` (required), `catalog: TocCatalogSp | null`, `allowedLevels: TocLevel[]`, `draft: SpAlignmentDraft` (required), `disabled`, `inlineErrors`, `catalogState: 'loading' | 'ready' | 'error'`.

Outputs: `draftChange = output<SpAlignmentDraft>()`, `retryCatalog = output<void>()`.

Computed: `spCode`, `alignsYes`, `hloFieldLabel` (label follows the level), `levelOptions`, `tocResultsForLevel`, `hloOptions` (AOW-prefixed label; title-only for EOI), `showEmptyHlo`, `selectedTocResult`, `indicatorOptions` (unfiltered), `selectedIndicator` (drives the contribution panel).

Change handlers (each emits a fresh draft with cascade resets): `onAlignsChange` (No nulls everything), `onLevelChange` (resets HLO/indicator/contribution), `onHloChange` (resets indicator/contribution), `onIndicatorChange`, `onContributionChange` (clamps $< 0 \rightarrow 0$), `onRetry`.

On “Yes” the block always shows a **determinate state**: loading / error+retry / per-(SP,level) empty / the cascade.

7. Key behaviors & engineering decisions

ID	Decision
Single save	Select + map + save in one PATCH (<code>sp_codes</code> + <code>toc_alignments</code> together). No pre-save step.
D-8a	The cascade selects use raw PrimeNG p-select / p-radioButton / p-inputNumber , not the wrapped <code>custom-fields/*</code> . The wrapped fields source options only from a registered <code>ControlListServices</code> and cannot take the in-memory, cascade-dependent catalog options. Styled to match via tokens.
D-6a	Destructive SP-deselect (an SP with a touched/saved alignment) confirms via <code>ActionsService.showGlobalAlert({severity:'delete', confirmCallback, cancelCallback})</code> — the repo’s actual house pattern (no <code>p-confirmDialog</code> exists).
D-10	The <code>toc_alignments</code> read-back is flat (snapshot fields on the row; no snapshot wrapper / <code>aow_code</code> / <code>is_stale</code>), matching the backend’s served shape. Staleness is derived FE-side by re-resolving <code>toc_result_id</code> against the live catalog.
Reveal-on-Yes fix	Selecting SPs populates <code>toc_drafts</code> via a deferred reconcile (<code>queueMicrotask</code>) — the wrapped multiselect emits <code>selectEvent</code> inside its own <code>formData.update()</code> , so a synchronous nested update was being clobbered. <code>onDraftChange</code> is an upsert so a “Yes” is never dropped.
Required per-SP answer	The per-SP ToC question is required (*): unanswered (<code>aligns_with_toc === null</code>) blocks Save and surfaces the Save-disabled hint (refines the earlier “unanswered = non-blocking” default).
Dropdown overflow	Body-appended select panels are capped + wrapped via a global <code>.sp-toc-select-panel</code> rule (<code>panelStyleClass</code>) so long indicator labels never overflow the viewport.
Chip enrichment	Selected SP chips resolve <code>code / allocation / icon_key</code> from the loaded science-programs list (<code>findScienceProgram</code>) so they render “SP02 — 30%” with the right icon.
Resilience	<code>WebsocketService</code> + <code>ClarityService</code> are defensively injected (degrade gracefully if absent); real-time <code>result.pool-</code>

ID	Decision
	<code>funding-alignment.changed</code> reconcile is dirty-state-guarded.

8. Tests

Co-located Jest specs; canonical fixtures in `testing/toc-catalog.fixture.ts` (SP01 live snapshot: OUTPUT/OUTCOME/EOI catalogs, saved-alignment + version-locked + empty-levels variants). No per-file re-mocking of catalog data.

Spec	Tests	Covers
<code>pool-funding-alignment.component.spec.ts</code>	101	N-blocks-per-N-SPs, per-SP independence (edit SP02 → SP06 untouched, in state + PATCH body), draft populate + upsert, reveal-on-Yes (no save), single-save body + pre-fill round-trip, 400 per-block routing, 409 version-lock, stale-snapshot render, deselect-confirm, required-answer gate + Save-disabled hint, carried gates (synced/PRMS/permission, <code>unknown_sp_codes</code>).
<code>sp-toc-alignment-block.component.spec.ts</code>	28	Yes/No reveal, level labels (§4.7), single-option level not preselected, AOW-prefixed/EOI labels, cascade resets, unfiltered indicators, contribution panel (2026 wording, read-only unit/target, ≥ 0), disabled/version-locked, catalogState loading/error/ready + retry, input-purity.
<code>bilateral.service.spec.ts</code>	(service block)	<code>getTocCatalog</code> envelope handling, <code>catalogForSp</code> , <code>draftsFromSaved / writeDtoFromDrafts</code> round-trip, <code>extractTocAlignmentErrors</code> tolerant parsing.
<code>api.service.spec.ts</code>	(bilateral)	<code>GET_PoolFundingHlosIndicators</code> typed to the catalog response.

Verification baseline: `npm run lint` clean, `npm run test` full suite green (5363 tests), AOT `tsc` clean, `jest.config.ts` coverage floors held. Microtask-timing assertions flush with `await Promise.resolve()`.

9. Removed / retired (for context)

The previous **HLO-selection-modal** flow was deleted when the inline cascade shipped: `all-modals/modals-content/hlo-selection-modal/`, `bilateral-action-card/`, `cache/hlo-selection-modal-context.service.ts`, the `hloSelection` modal registration, the modal-era types (`BilateralHlosIndicatorsResponse`, `BilateralHlosPair`, `IndicatorRow`, `HloMapping`, `PrmsToc*`, ...), and the `bilateral.fixtures.ts` mocks. A grep gate enforces their absence (`aow_status|no_aow_mappings|BilateralHlosPair|HloSelectionModal|hloSelection|areaOfWork`).

10. Open items / follow-ups (not blocking)

- **Live golden-path sign-off** of the catalog flow against the deployed testing environment (gated on the backend deploy). Local verification against a running backend passed (contract match + cascade from real lambda-toc data).
- **Catalog-503 robustness:** a top-level `hlos-indicators` cold-cache 503 leaves `allowed_levels` unknown, so the ToC section + its block-level retry currently hide silently; a small page-level error/retry banner is recommended.

All bilateral SDD specs (requirements/design/tasks/execution + the design-decision records D-8a/D-6a/D-10) are archived under `docs/specs/archive/2026-06-17-bilateral-module*` for the full decision trail.